1

对字符开桶，令 `c[x]` 表示 T 中字符 x 的出现次数**减去当前窗口中该字符的出现次数**。初始时遍历 T，令 `c[T[i]]++`。

用双指针 i, j 维护窗口 $S[i \dots j]$。右移 j 时令 `c[S[j]]--`，当所有 $c[x] \le 0$ 时，说明当前窗口已覆盖 T，此时尝试右移 i 缩小窗口，并更新最短子串。

为避免每次检查所有桶，额外维护 bad 表示满足 $c[x] > 0$ 的字符种类数。当某个字符的计数由 1 变为 0 时，`bad--`；由 0 变为 1 时，`bad++`。因此只需判断 `bad == 0` 即可确定当前窗口是否合法。

```
pair<string, int> mincover(const string& S, const string& T){
    if(T.empty()) return {"", 0};

    int c[256] = {}, bad = 0;
    for(unsigned char x : T){
        if(c[x] == 0) bad++;
        c[x]++;
    }

    int i = 0, st = -1, n = S.size(), len = n + 1;
    for(int j = 0 ; j < n ; j++){
        unsigned char x = S[j];
        if(c[x] == 1) bad--;
        c[x]--;

        while(!bad){
            if(j - i + 1 < len){
                len = j - i + 1;
                st = i;
            }
            unsigned char y = S[i++];
            if(!c[y]) ++bad;
            c[y]++;
        }
    }
    if(st == -1) return {"", -1};
    return {S.substr(st, len), st};
}
```

2

先对 P 构造 next 数组。

用 i, j 分别表示 T 的扫描位置和 P 中已匹配的字符数。若字符相同，则同时右移；否则令 `j = next[j]`。

当 $j = m$ 时，说明找到一次完整匹配，记录起始下标 $i - m$。由于允许重叠，匹配成功后令 `j = next[j]`，去找下一次匹配。

```
vector<int> allind(const string& P, const string& T){
    vector <int> ans;
```

```
    int m = P.size(), n = T.size();
    if(m == 0) return ans;

    vector <int> nxt(m + 1);
    nxt[0] = -1;
    for(int i = 0, j = -1 ; i < m ; ){
        if(j == -1 || P[i] == P[j])
            nxt[++i] = ++j;
        else j = nxt[j];
    }

    for(int i = 0, j = 0 ; i < n ; ){
        if(j == -1 || T[i] == P[j]){
            i++, j++;
            if(j == m){
                ans.push_back(i - m);
                j = nxt[j];
            }
        }
        else j = nxt[j];
    }
    return ans;
}
```

3

记 `l = n - next[n]`，则

- 若 $n = 0 \pmod{l}$，则答案为 l；
- 否则答案为 n。

```
int cycle(const string& S){
    int n = S.size();
    if(n == 0) return 0;
    vector <int> nxt(n + 1);
    nxt[0] = -1;
    for(int i = 0, j = -1 ; i < n ; ){
        if(j == -1 || S[i] == S[j])
            nxt[++i] = ++j;
        else j = nxt[j];
    }

    int l = n - nxt[n];
    return n % l == 0 ? l : n;
}
```

4

考虑维护左侧未被 `c` 匹配到的、且与当前位置直接相连的字符 `a` 的数量 c_a。

- 当扫到 `b`，直接删除并丢弃；
- 当扫到 `a`，将 $c_a \leftarrow c_a + 1$。
- 当扫到 `c`，若 $c_a \geq 1$，则 $c_a \leftarrow c_a - 1$，并删除 `c`；否则将 `c` 插入答案串中。
- 当扫到其他字符 h，先插入 c_a 个 `a` 到答案串中，再插入 h 到答案串中，令 $c_a \leftarrow 0$。

```
int deleta(string& S){
    int n = S.size(), ca = 0, w = 0;
    for(int i = 0 ; i < n ; i++){
        char ch = S[i];
        if(ch == 'b') continue;
        else if(ch == 'a') ca++;
        else if(ch == 'c'){
            if(ca) ca--;
            else S[w++] = 'c';
        }
        else{
            for(int j = 0 ; j < ca ; j++)
                S[w++] = 'a';
            S[w++] = ch;
            ca = 0;
        }
    }
    for(int j = 0 ; j < ca ; j++)
        S[w++] = 'a';
    return w;
}
```